

AI Customer Support Agent (RAG + Memory) Technical Report

Course: Generative AI

Student: Esraa Wael Al Shami

Project: Jordan Clothing AI Customer Support Agent

Doctor: Abeer Alhyari

Page 1 — Overview

The goal of this project is to build an AI customer support agent for a Jordanian online clothing store. The agent answers common support questions using Retrieval-Augmented Generation (RAG), follows strict store policies, supports Arabic and English, and uses conversation memory to stay consistent across multi-turn chats. No fine-tuning was used; the system relies on prompting, retrieval, and rule-based safeguards.

Scenario and Target Users

Scenario 5: AI Customer Support Agent (RAG + Memory). Primary users are end customers. A secondary agent-assist feature is included to help customer support agents by providing conversation summaries and escalation context.

Assumptions and Constraints

The system assumes an online-only clothing store. Returns are allowed within 48 hours only for valid reasons, while discounted items are non-returnable. Orders cannot be cancelled once they are out for delivery. The system relies on pre-trained language models without fine-tuning and enforces policies through RAG and rule-based checks. Sensitive data is not collected, and unclear cases are handled through escalation.

User Flow (5–7 steps)

1. The user opens the chat and starts a session.
2. The user sends a question (Arabic or English).
3. The system classifies the message (intent, tone, language, and entities such as order ID).
4. The policy engine checks whether a strict rule applies (policy override).
5. If no override applies, the system retrieves the most relevant FAQs/policies from the RAG knowledge base (top-k).
6. The system generates a safe response using retrieved evidence and conversation memory, then updates the memory summary.

7.If confidence is low, evidence is missing, or the request is sensitive/out of scope, the system escalates the case and asks for safe required details.

Page 2 — System Design

The system is modular and follows a typical enterprise design: UI/API input, prompt templates, model layer, retrieval layer, safety layer (policy + escalation), evaluation layer, and logging. The design reduces hallucination by grounding answers in retrieved documents and enforcing rules before generation.

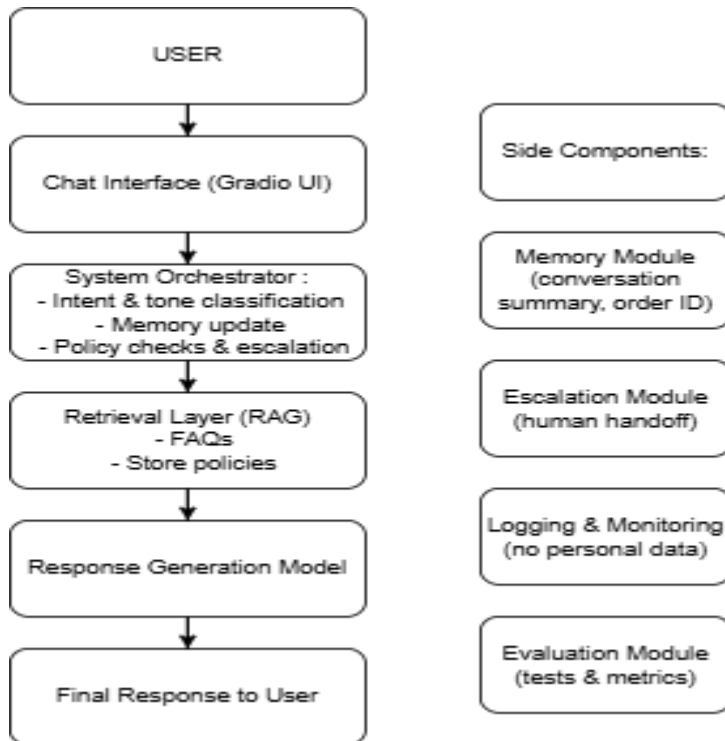


Figure 1 :System architecture showing main data flow and supporting components

Data Flow (and where escalation happens)

1. The user message is received through the chat interface.
2. A classifier analyzes the message to detect intent, language, tone, and key entities.
3. The policy engine applies strict store rules first and blocks any disallowed actions.
4. If allowed, the retriever queries the vector index and returns the top-k relevant FAQ and policy documents.
5. The generation model produces a grounded response using the retrieved documents and conversation memory.
6. Conversation memory is updated with key facts and a short summary.

7.If confidence is low, evidence is missing, or sensitive data is detected, the system escalates the case instead of generating an answer.

Page 3 — Prompting + Escalation

Prompt Engineering (3 strategies)

- Role-based prompting: The system is instructed to act as a professional customer support agent for a Jordanian online clothing store, with a clearly defined scope.
- Policy constraints: Store policies are enforced as hard rules in the prompt and cannot be overridden by memory or user requests.
- Uncertainty handling: When confidence is low or information is missing, the system escalates the case instead of guessing.

Mini prompt iteration (Before → After)

Before:

“Answer the customer politely and try to help.”

After:

“Answer as a customer support agent for a Jordanian online clothing store. Use only retrieved FAQ and policy information. Follow store policies strictly. Do not approve returns for discounted items. If information is missing or confidence is low, escalate instead of guessing.”

Escalation Triggers and Paths

Trigger	Detection	System action	User message (short)
Sensitive data (OTP/card/CVV/password)	Regex / pattern match	Escalate + refuse to process	Please do not share sensitive data. Provide order/shipment ID only.
Low intent confidence	Classifier confidence < 0.60	Escalate or ask clarification	I need one more detail to help correctly (order ID / issue).

No retrieval evidence	Top-k empty or irrelevant	Escalate	I cannot confirm from our knowledge base; I will escalate to support.
Out of scope	Classifier + patterns	Escalate	This request is outside store support; I will forward it to a human agent.
High negative tone / repeated issue	Tone=angry/frustrated	Escalate earlier	I understand. I will escalate this to resolve it quickly.

Table 1: Escalation triggers and system actions.

Model comparison and final selection

Aspect	GPT-4o	GPT-4o-mini
Typical use in this project	Final response generation (production)	Baseline / low-cost prototyping
Accuracy & reasoning	Very high	Lower than GPT-4o
Policy adherence	Excellent (more stable on edge cases)	Good but weaker on difficult policy cases
Arabic support	Strong (better phrasing + consistency)	Moderate
Latency	Low–medium	Very low (fast)
Cost	Medium	Low; can be used through gateways like OpenRouter (often with low-cost credits)
Safety / hallucination risk	Lower	Higher than GPT-4o

Table 2: Model comparison (GPT-4o vs GPT-4o-mini).

Final choice: GPT-4o was selected because it showed stronger policy compliance and more reliable answers in Arabic. GPT-4o-mini is useful for fast testing and lower cost runs, but it is less accurate in policy-sensitive cases.

Data preparation pipeline

The project uses two main data sources. First, a public dataset of customer support conversations from Twitter was used to learn common support patterns and create realistic test scenarios. Second, a bilingual (Arabic/English) FAQ and policy knowledge base was built from real clothing-store support content and then adapted to match the Jordanian market constraints (for example: delivery fees in JOD, 48-hour returns, and non-returnable discounted items). The final RAG knowledge base was stored as Q/A documents and indexed for retrieval.

Pipeline Steps (6–8 steps)

1. Load the raw customer support dataset (CSV) and inspect columns and missing values.
2. Filter inbound customer messages and match them with the corresponding support replies to form Q/A pairs.
3. Remove duplicates and noisy entries (empty text, repeated questions, and broken rows).
4. Apply domain filtering to keep retail and clothing-related conversations only.
5. Collect real store policies and FAQ topics, then rewrite them into clear bilingual Q/A documents aligned with the Jordanian store rules.
6. Normalize text (basic cleaning, consistent formatting, language tags, and category labels such as returns, delivery, orders).
7. Create embeddings for each FAQ/policy document using a sentence embedding model, then build a FAISS vector index.
8. Split usage into: RAG knowledge base (retrieval), evaluation questions (tests), and a small holdout set for final checks.

Page 5 — Evaluation + Reliability + Delivery

Evaluation plan (10–20 tests grouped by category)

Category	Examples (tests)	Pass criteria
Policy compliance	Discount return refused; 48-hour rule; no cancel out-for-delivery	Answer matches policy and does not contradict rules
RAG grounding	Answer uses retrieved policy/FAQ content	No unsupported claims; uses retrieved evidence
Intent accuracy	Delivery vs cancellation vs returns	Classifier intent matches the question
Bilingual support	Same query in Arabic and English	Correct language + correct policy
Memory	Order ID mentioned then reused later	Order ID appears in later response when needed
Escalation	Sensitive info request; low confidence; no docs	Escalation message shown + no sensitive data requested

Table 3: Evaluation plan summary.

Failure cases (≥ 3) and mitigations

Failure case	Root cause	Mitigation
Discount item incorrectly approved for return	RAG retrieved a generic 'defect return' answer	Policy override checks discount + return/exchange and blocks
Short/unclear question causes wrong intent	Not enough context for intent	Confidence threshold + clarification/escalation
Model invents policy detail (hallucination)	Generation without strong grounding	RAG-only evidence + escalation when evidence is missing

Table 4: Failure cases and mitigations.

Optimization step (evidence)

Observed improvement: adding the policy override (discount + return/exchange) reduced incorrect approvals in the test prompts. Before: discounted-return questions could match a defect-return answer. After: policy override blocks the return and returns the correct policy response.

Reproducibility (how to run)

The project is designed to be fully reproducible using a single Google Colab notebook. All dependencies are installed at runtime using standard Python package managers. The system is organized into clear modules for data loading, retrieval (RAG), policy enforcement, memory handling, and the user interface.

To run the project, the user opens the Colab notebook, installs the required libraries, sets the API key using environment variables or Colab Secrets, and executes the cells in order. The Gradio interface is then launched directly from the notebook. A README file documents the setup steps, directory structure, and execution order to ensure consistent results across different environments.

Limitations + ethics risks/mitigations

The system is limited by the size and diversity of the available knowledge base. Some user questions may be phrased in uncommon dialects or use wording not covered by the FAQs, which can reduce retrieval accuracy. These cases are handled through escalation rather than speculative answers.

From an ethical and privacy perspective, the system is designed to avoid collecting or storing sensitive personal data such as payment details, passwords, or one-time passwords. Any request involving such information triggers escalation.

The main technical risk is hallucination, where a language model may generate incorrect policy details. This risk is mitigated by grounding responses in retrieved documents, enforcing strict policy overrides, and escalating cases when evidence is missing or confidence is low.

References (IEEE)

- [1] Thought Vector, “Customer Support on Twitter,” Kaggle Dataset.
- [2] [2] MAX Fashion Help Center, “Returns Policy / FAQs,” Help Center pages.
- [3] Meta AI Research, “FAISS: A library for efficient similarity search,”
- [4] Gradio, “Gradio Documentation,” .
- [5] OpenAI, “Chat Completions API Reference,” .
- [6] Sentence-Transformers, “all-MiniLM-L6-v2,” Hugging Face Model Card.
- [7] OpenRouter, “openai/gpt-4o-mini,” Model page.
- [8] OpenAI, “GPT-4o mini and GPT-4o model documentation,” .